Desarrollo con IA Generativa

Modelo de IA seleccionado

La herramienta de desarrollo que he seleccionado es Claude con Sonnet 4.6 porque me pareció la mejor opción dentro de las IA que conozco. Las otras dos opciones eran ChatGPT y Gemini, y el proceso de descarte fue darles el mismo prompt y ver cómo respondian y que necesitaban para seguir desarrollando el proyecto, y en este caso Claude es la que tiene mejores respuestas y más claridad en el desarrollo. A pesar de que daba respuestas claras, yo tuve que redirigirla en varias decisiones que tomaba, para que el proyecto fuera como yo lo había visionado.

Registro de las interacciones con la IA

La interacción más importante que tuve fue el prompt inicial que es el que cimienta las bases del proyecto que quería desarrollar:

> necesito que me ayudes a crear un front end para una aplicacion web con back end en laravel. La web consiste en una api de creacion de eventos para hacer meetup y jugar partidos de futbol. Podriamos empezar por los enpoints que son publicos y todo el mundo puede acceder. El primer endpoint seria el indice, donde se montrarian los partidos creados por los usuarios. Me gustaría que esté escrito en React y que el endpoint de laravel "/matches" se conecte con esta vista.

La IA desde el primer momento ya me ayudó a crear el proyecto indicando cómo instalar todo, como yo ya había tocado React con Vite, me hizo varias preguntas de configuración del proyecto y decidí desarrollarlo con React Router para las rutas.

Desde el comienzo la IA me hizo usar datos Mock para desarrollar, pero le tuve que insistir en que yo quería conectarlo todo con el Backend que ya estaba levantado y funcionando para ver que cada paso que íbamos dando funcionara correctamente. Así me ayudó a configurar el archivo *vite.config.js* para conectar el Front con el Back, además me empezó a pedir que formato devolvió la api (JSON) y qué devolvía exactamente para poder mostrarlo correctamente.

prefiero ir conectando todo y ver que funciona bien, sin mocks

Tiene sentido. Entonces necesitamos asegurarnos de que Laravel responde bien antes de probar el front. Dime:

P: ¿Tienes Laravel ya levantado y corriendo?
R: Sí, en localhost:8000

P: ¿Tienes ya el endpoint /api/matches creado en Laravel?
R: Sí, y devuelve datos

Perfecto. Entonces el primer paso es verificar qué formato exacto devuelve el endpoint.

Yo le indiqué qué paleta de colores usar, ya que al principio la IA me quería hacer usar unos colores que no iban de acuerdo con la idea que tenía de la web.

perfecto, muestra los partidos que hay en la bbdd, ahora, te comento que los colores que me has puesto no me convencen, te paso una color palette para cambiar el css: https://colorhunt.co/palette/25671e48a111f2b50bf7f0f0

Failed to fetch: https://colorhunt.co/palette/25671e48a111f2b50bf7f0f0 >

No problem, el código de la URL me da los colores directamente:

- `#25671E` — Verde oscuro
- `#48A111` — Verde vibrante
- `#F2B50B` — Amarillo/naranja
- `#F7F0F0` — Blanco cálido

Al principio me quedaba sin *Tokens* muy rápido y no entendía bien el porqué, no me dejaba interactuar más hasta pasado muchas horas. Hasta que entendí que el CSS que me generaba hacía que se terminaran más rápido el uso de interacción que podía tener. Fue así que decidí preguntarle sobre este tema y me dió una muy buena solución, que luego entendí que era el estándar de programar en React que es el uso de CSS Modules. De esta manera pude seguir haciendo más prompts, centrarme en la parte más técnica y una vez funcionando pedirle que me hiciera el CSS y ponerlo en el archivo separado de la lógica.

antes de seguir una pregunta, me es muy incomodo poner en cada componente estilos en ese componente, no hay una manera mas comoda y limpia?

Sí, totalmente. La solución más limpia para este proyecto es un **archivo CSS global** con todas las variables y estilos compartidos, y opcionalmente **CSS Modules** para estilos específicos de cada componente.

La opción más práctica para lo que tenemos es un único `index.css` global con las variables de la paleta y los estilos base, y luego un archivo `.css` por componente importado directamente. Así:

Una vez creada toda la aplicación web, ya sólo quedaban detalles que le tenía que ir indicando, como por ejemplo: En el caso de dispositivos móviles el menú no se veía bien con el tamaño de los botones, así que le indiqué cambiarlo a estilo hamburguesa.

hecho lo de index.css, UserProfile al final no lo pusimos acuerdate.
Hay un detalle en el navbar, cuando se hace pequeño a tamaño de movil, sejunta mucho el nombre de la api, el nombre de usuario y los botones, me gustaria que en moviles fuera un menu desplegable, se puede hacer?

3 abr

Claro, es un patrón muy común. Lo haré con un botón hamburguesa que abre un menú desplegable en móvil, sin ninguna librería extra. Pásame tu `Navbar.jsx` actual y lo actualizo directamente.

En la parte final de las interacciones, sólo eran prompts muy cortos para terminar de pulir la aplicación.

ahora me gustaria saber si se puede hacer un modal para que cuando el admin elimine el partido en vez de un alert salga el modal en cuestion

13:11

Sí, perfectamente. Como el modal lo usaremos solo en `MatchDetail`, lo creo directamente como componente dentro del mismo archivo. Añade el componente `ConfirmModal` y actualiza `handleDelete` en tu `MatchDetail.jsx`:

Análisis del código generado por la IA

Había momentos en que la IA no acababa de entender algunos detalles, como por ejemplo: En la aplicación solo los usuarios registrados podían ver a los jugadores que se habían registrado. La IA habia dejado ese campo aunque no estuvieras autentificado y lo enseñaba con un mensaje de “No hay jugadores registrados” aunque habían jugadores registrados, en este caso la solución fué añadir una línea. Tienes que estar atento a estos pequeños detalles si quieres tener la aplicación como tu la visionamos.

```
{isAuth && (
  <div className={styles.section} style={{ animationDelay: "60ms" }}>
    <div className={styles.sectionTitle}>Jugadores</div>
    <PlayersList matchId={id} token={token} />
  </div>
)}
```

Descripción del proceso de conexión entre el Frontend y el Backend

Tuve que configurar el archivo *vite.config.js* configurando el proxy y apuntado hacia la dirección donde está la API levantada que en este caso es http://localhost:8000. Luego levantar tanto el servicio Backend como el servicio Frontend.

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:8000",
        changeOrigin: true,
      },
    },
  },
});
```

Reflexión sobre el proceso de aprendizaje

Fue interesante construir desde cero un proyecto a través de la IA. Siento que al comienzo debes darle detalles de todo, para que de esta manera sepa qué hacer con exactitud. Uno debe corregir lo que hace. Más adelante pasas a una fase donde ya sabe qué hacer y va más rápido y fluido. En la fase final tienes que pulir los detalles para que la aplicación quede perfecta, controlando los pequeños fallos/ajustes que tiene la IA frente a lo que uno tiene en mente del proyecto global.